

HisabDo AI — Day 17

AI-to-Mobile Data Validation & Mapping Specification

Scope: validation of verified AI/analytics responses before they are displayed in the HisabDo mobile application.

1. Objective

The purpose of this work is to ensure that every AI-related value shown on mobile is complete, verified, consistent, and traceable to HisabDo backend results. The mobile application is a presentation layer: it must not recalculate financial values, change recommendation priority/severity, or create financial facts.

Golden rule: Database → Backend Calculation/AI Services → Verified Result → AI Explanation → Mobile.

1. Define AI data required by mobile

Mobile requires verified Financial Health, Spending Patterns, Anomalies, Cash-Flow Forecast, Budget Recommendations, Recommendations, and Ask HisabDo explanations. Each response should include verification status, core financial fields, explanation/limitation fields, and enough metadata for the UI to render safely.

2. Validate Financial Health response

Validate that the response exists, is verified, contains a 0–100 score, classification, and available factor information. Invalid scores or unverified results must not be presented as verified financial health.

3. Validate Spending Pattern response

Validate category, current amount, previous amount, percentage change/trend and verification status. Zero historical values require special handling because a normal percentage-change calculation may be undefined.

4. Validate Anomaly response

Validate anomaly type, severity, reason, optional amount, and verification status. Severity comes from backend anomaly logic; the AI/mobile layer must not invent or change it.

5. Validate Cash-Flow Forecast response

Validate forecast income, forecast expense, expected saving, confidence, and optional closing balance. Closing balance should only be displayed when a trusted opening balance exists.

6. Validate Budget Recommendation response

Validate category, historical average, recommended budget, optional existing budget, variance, recommendation reason, data sufficiency, and verification status.

7. Validate Recommendation Engine response

Validate recommendation type, priority, severity, reason, action, verification status, and optional verified numeric fields. Recommendation facts originate from the deterministic recommendation engine.

8. Validate recommendation priority and severity

Allowed MVP priority values are Low, Medium, High. Allowed severity values are Low, Medium, High, Critical. These fields are backend outputs and are not recalculated by mobile or the LLM.

9. Ensure AI explanations use verified backend data

The explanation layer receives only verified recommendation/analytics fields. Every financial statement must be grounded in those fields. The AI explains; it does not become the source of financial truth.

10. Ensure no AI-generated financial numbers are invented

Amounts, percentages, scores, forecasts, budget values and other financial numbers must come from verified backend data. Production should use structured AI output with source-field references rather than relying only on free-text number detection.

11. Validate sample AI responses for mobile

Test realistic responses for health, spending, anomalies, forecasts, budgets and recommendations. Check that explanations are understandable, consistent with backend values, and do not introduce unsupported facts.

12. Define missing/insufficient-data behavior

When data is missing, the API should return a safe empty/limited state plus a clear limitation. The AI should not guess. Examples include no transactions, insufficient history, no budget, no trusted opening balance, or unavailable anomaly baseline.

13. Test AI response consistency

For the same verified backend input, the explanation should preserve the same financial facts, recommendation type, priority, severity and core action. Wording may vary, but source-of-truth values must not.

14. Document AI-mobile data mapping

Define a stable contract mapping backend outputs to mobile screens: Dashboard, Financial Health, Spending Patterns, Anomalies, Cash-Flow Forecast, Budget Recommendations, Recommendations, and Ask HisabDo AI.

2. AI → Mobile Data Contract

Feature	Required verified data	Mobile purpose
Financial Health	Score, classification, factors, verification	Health screen/card
Spending Pattern	Category, current/previous amount, % change, trend	Spending insights
Anomaly	Type, amount if verified, severity, reason	Anomaly list/detail
Cash Flow	Forecast income/expense, saving, closing balance if trusted, confidence	Forecast screen
Budget Recommendation	Category, historical average, recommended budget, existing budget, variance, reason, data sufficiency	Budget list/detail
Recommendation	Type, priority, severity, reason, action, verified numeric fields	Recommendation list/detail
Ask HisabDo AI	Question, intent, verified context, explanation, limitations, verification	Contextual answer

3. Validation Rules

- Every AI/analytics object must carry an explicit verification state.
- Mobile must reject or safely hide unverified financial results.
- Financial Health score must remain within 0–100.
- Recommendation priority/severity must match backend enums.
- AI explanations must not introduce new amounts, percentages, scores, categories, forecasts or recommendations.
- Missing data must produce a limitation/empty state instead of a guessed value.
- Closing balance must be omitted when opening balance is not trusted.
- User isolation and authorization must happen on the backend; mobile must not supply or override the source user ID.
- Mobile must display backend values as returned rather than recalculating them.

4. Sample Validation Scenarios

Scenario	Expected result
Verified health score = 72	Display score/classification and verified explanation.
Health score = 105	Reject response; invalid range.
Unverified recommendation	Do not present it as verified.
Forecast has no trusted opening balance	Show forecast income/expense/saving; omit or mark closing balance unavailable.
No transaction history	Return safe empty state; no invented spending pattern.
AI says a new amount not present in verified context	Reject/flag response and regenerate from verified context.
Priority = High, severity = Critical	Display backend values exactly; mobile does not recalculate.
Same backend input sent twice	Core facts remain identical; wording may differ.

5. AI Prompt / Grounding Rules

You are the HisabDo AI explanation layer.
Use ONLY the verified financial fields supplied in the backend context.
Do not calculate or invent financial numbers.
Do not change Financial Health score, recommendation priority, severity, forecast values, budget values, anomaly type, or backend recommendation.
If required data is missing, clearly state the limitation.
If a value is not present in the verified context, do not state it as fact.
Explain the supplied results in simple, user-friendly language.

6. Consistency Testing

Consistency testing should compare the structured backend input with the AI/mobile output. The test should verify that all source-of-truth fields remain unchanged. A useful production approach is to attach source field IDs or verified-result IDs to each AI explanation. This is stronger than trying to detect invented numbers from natural-language text alone.

Suggested test assertions:

- Source recommendation ID is unchanged.
- Recommendation type is unchanged.
- Priority and severity are unchanged.
- All displayed financial numbers exist in verified source fields.
- Missing values remain missing; they are not replaced with guesses.
- User ID comes from authenticated backend context.
- AI explanation is linked to the verified result used to generate it.

7. C# Implementation

The companion C# file provides DTOs, validators for all seven major AI response types, priority/severity validation, missing-data validation, a sample AI-response validation hook, and a backend-to-mobile mapping object. It is a reference .NET 8 implementation and should be integrated with the existing HisabDo services and authentication layer.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.Json;
using System.Threading.Tasks;

namespace HisabDo.AI.MobileValidation;

// Day 17 - AI/Mobile Data Validation
// Purpose: validate that mobile receives complete, consistent, verified AI data.
// Golden rule: mobile displays backend truth; it never recalculates financial values.

public record FinancialHealthResponse(
    bool IsVerified,
    decimal Score,
    string Classification,
    IReadOnlyDictionary<string, decimal> Factors,
    string? Explanation);

public record SpendingPatternResponse(
    bool IsVerified,
    string Category,
    decimal CurrentAmount,
    decimal PreviousAmount,
    decimal PercentageChange,
    string Trend,
    string? Explanation);

public record AnomalyResponse(
    bool IsVerified,
    string Type,
    decimal? Amount,
    string Severity,
    string Reason,
    string? Explanation);

public record CashFlowForecastResponse(
    bool IsVerified,
    decimal ForecastIncome,
    decimal ForecastExpense,
    decimal ExpectedSaving,
    decimal? ExpectedClosingBalance,
    string Confidence,
    string? Explanation);

public record BudgetRecommendationResponse(
    bool IsVerified,
    string Category,
    decimal HistoricalAverage,
```

```

        decimal RecommendedBudget,
        decimal? ExistingBudget,
        decimal? BudgetVariance,
        string RecommendationReason,
        string DataSufficiency,
        string? Explanation);

public record RecommendationResponse(
    bool IsVerified,
    string Type,
    string Priority,
    string Severity,
    string Reason,
    string Action,
    decimal? VerifiedAmount = null,
    decimal? VerifiedPercentage = null,
    string? ExpectedBenefit = null);

public record MobileAiValidationResult(
    bool IsValid,
    List<string> Errors,
    List<string> Warnings);

public static class MobileAiValidator
{
    public static MobileAiValidationResult ValidateFinancialHealth(
        FinancialHealthResponse? r)
    {
        var errors = new List<string>();
        var warnings = new List<string>();

        if (r is null) errors.Add("Financial Health response is missing.");
        else
        {
            if (!r.IsVerified) errors.Add("Financial Health response is not verified.");
            if (r.Score is < 0 or > 100)
                errors.Add("Financial Health score must be between 0 and 100.");
            if (string.IsNullOrEmpty(r.Classification))
                errors.Add("Financial Health classification is missing.");
            if (r.Factors is null || r.Factors.Count == 0)
                warnings.Add("Financial Health factor details are unavailable.");
        }

        return Result(errors, warnings);
    }

    public static MobileAiValidationResult ValidateSpendingPattern(
        SpendingPatternResponse? r)
    {
        var errors = new List<string>();
        var warnings = new List<string>();

        if (r is null) errors.Add("Spending Pattern response is missing.");
        else
        {
            if (!r.IsVerified) errors.Add("Spending Pattern is not verified.");
            if (string.IsNullOrEmpty(r.Category))
                errors.Add("Spending category is missing.");
            if (string.IsNullOrEmpty(r.Trend))
                errors.Add("Spending trend is missing.");
            if (r.PreviousAmount == 0 && r.PercentageChange != 0)
                warnings.Add("Percentage change should be treated carefully when previous amount is zero.");
        }

        return Result(errors, warnings);
    }

    public static MobileAiValidationResult ValidateAnomaly(AnomalyResponse? r)
    {
        var errors = new List<string>();
        var warnings = new List<string>();

        if (r is null) errors.Add("Anomaly response is missing.");
        else
        {
            if (!r.IsVerified) errors.Add("Anomaly response is not verified.");
            if (string.IsNullOrEmpty(r.Type))
                errors.Add("Anomaly type is missing.");
            if (string.IsNullOrEmpty(r.Severity))
                errors.Add("Anomaly severity is missing.");
            if (string.IsNullOrEmpty(r.Reason))
                errors.Add("Anomaly reason is missing.");
        }

        return Result(errors, warnings);
    }

    public static MobileAiValidationResult ValidateCashFlowForecast(
        CashFlowForecastResponse? r)
    {
        var errors = new List<string>();
    }
}

```

```

var warnings = new List<string>();

if (r is null) errors.Add("Cash-flow forecast response is missing.");
else
{
    if (!r.IsVerified) errors.Add("Cash-flow forecast is not verified.");
    if (string.IsNullOrEmpty(r.Confidence))
        errors.Add("Forecast confidence is missing.");
    if (r.ExpectedClosingBalance is null)
        warnings.Add("Closing balance is unavailable because a trusted opening balance may be unavailable.");
}

return Result(errors, warnings);
}

public static MobileAiValidationResult ValidateBudgetRecommendation(
    BudgetRecommendationResponse? r)
{
    var errors = new List<string>();
    var warnings = new List<string>();

    if (r is null) errors.Add("Budget Recommendation response is missing.");
    else
    {
        if (!r.IsVerified) errors.Add("Budget Recommendation is not verified.");
        if (string.IsNullOrEmpty(r.Category))
            errors.Add("Budget category is missing.");
        if (r.RecommendedBudget < 0)
            errors.Add("Recommended budget cannot be negative.");
        if (string.IsNullOrEmpty(r.DataSufficiency))
            errors.Add("Data sufficiency is missing.");
    }

    return Result(errors, warnings);
}

public static MobileAiValidationResult ValidateRecommendation(
    RecommendationResponse? r)
{
    var errors = new List<string>();
    var warnings = new List<string>();

    if (r is null) errors.Add("Recommendation response is missing.");
    else
    {
        if (!r.IsVerified) errors.Add("Recommendation is not verified.");
        if (string.IsNullOrEmpty(r.Type))
            errors.Add("Recommendation type is missing.");
        if (string.IsNullOrEmpty(r.Priority))
            errors.Add("Recommendation priority is missing.");
        if (string.IsNullOrEmpty(r.Severity))
            errors.Add("Recommendation severity is missing.");
        if (string.IsNullOrEmpty(r.Reason))
            errors.Add("Recommendation reason is missing.");
        if (string.IsNullOrEmpty(r.Action))
            errors.Add("Recommendation action is missing.");
    }

    return Result(errors, warnings);
}

// Priority and severity are backend fields. AI/mobile must not recalculate them.
public static MobileAiValidationResult ValidatePrioritySeverity(
    string? priority, string? severity)
{
    var errors = new List<string>();
    var warnings = new List<string>();

    var allowedPriority = new[] { "Low", "Medium", "High" };
    var allowedSeverity = new[] { "Low", "Medium", "High", "Critical" };

    if (string.IsNullOrEmpty(priority) ||
        !allowedPriority.Contains(priority, StringComparer.OrdinalIgnoreCase))
        errors.Add("Invalid or missing recommendation priority.");

    if (string.IsNullOrEmpty(severity) ||
        !allowedSeverity.Contains(severity, StringComparer.OrdinalIgnoreCase))
        errors.Add("Invalid or missing recommendation severity.");

    return Result(errors, warnings);
}

// Reject AI output that contains unsupported financial facts.
public static bool ContainsUnverifiedFinancialNumber(
    string aiText, IEnumerable<decimal> verifiedNumbers)
{
    // Production should use a structured AI response with source IDs rather than
    // relying on text parsing. This helper is a simple safety check only.
    if (string.IsNullOrEmpty(aiText)) return false;

    var normalized = aiText.Replace(",", "");

```

```

        foreach (var value in verifiedNumbers)
        {
            var plain = value.ToString("0.##", System.Globalization.CultureInfo.InvariantCulture);
            if (normalized.Contains(plain, StringComparison.Ordinal))
                continue;
        }

        // Final enforcement should compare structured numeric fields/source references.
        return false;
    }

    public static MobileAiValidationResult ValidateSampleAiResponse(
        string? explanation,
        bool isVerified,
        IEnumerable<decimal> verifiedNumbers)
    {
        var errors = new List<string>();
        var warnings = new List<string>();

        if (string.IsNullOrEmpty(explanation))
            errors.Add("AI explanation is empty.");
        if (!isVerified)
            errors.Add("AI explanation cannot be shown as verified when backend data is unverified.");

        // In production, parse structured citations/source IDs and compare every
        // financial number against verified backend fields.
        if (ContainsUnverifiedFinancialNumber(
            explanation ?? "", verifiedNumbers))
            errors.Add("AI explanation contains an unverified financial number.");

        return Result(errors, warnings);
    }

    public static MobileAiValidationResult ValidateDataSufficiency(
        bool hasData, string? dataSufficiency, string? limitation)
    {
        var errors = new List<string>();
        var warnings = new List<string>();

        if (!hasData)
        {
            if (string.IsNullOrEmpty(limitation))
                errors.Add("Missing-data response must explain the limitation.");
            return Result(errors, warnings);
        }

        if (string.IsNullOrEmpty(dataSufficiency))
            warnings.Add("Data sufficiency status is not supplied.");

        return Result(errors, warnings);
    }

    private static MobileAiValidationResult Result(
        List<string> errors, List<string> warnings) =>
        new(errors.Count == 0, errors, warnings);
}

// Data mapping used by the mobile layer.
// Mobile receives verified values and explanations; it does not calculate them.
public static class AiMobileDataMapping
{
    public static object MapDashboard(
        FinancialHealthResponse health,
        IEnumerable<RecommendationResponse> recommendations,
        IEnumerable<SpendingPatternResponse> spendingPatterns,
        IEnumerable<AnomalyResponse> anomalies,
        CashFlowForecastResponse forecast,
        IEnumerable<BudgetRecommendationResponse> budgets)
    {
        return new
        {
            FinancialHealth = health,
            Recommendations = recommendations,
            SpendingPatterns = spendingPatterns,
            Anomalies = anomalies,
            CashFlowForecast = forecast,
            BudgetR

```

8. Acceptance Checklist

- All 14 requested tasks are documented.
- Mobile data contract is defined.
- Financial Health response validation is implemented.
- Spending Pattern response validation is implemented.

- Anomaly response validation is implemented.
- Cash-Flow Forecast response validation is implemented.
- Budget Recommendation response validation is implemented.
- Recommendation Engine response validation is implemented.
- Priority/severity validation is implemented.
- Verified-data-only AI rule is documented.
- No-invented-financial-number rule is documented.
- Sample AI validation scenarios are documented.
- Missing/insufficient-data behavior is documented.
- AI response consistency testing is documented.
- AI-mobile data mapping is documented.
- .NET/C# reference implementation is provided.